MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 1 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Instrucțiuni liniare

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Citirea unei valori reale x și calcularea expresiei y = (sin x + 2x) / (x³ − 2x). Demonstrarea matematică a rezultatului și analiza domeniului de definiție.

Material teoretic

Un algoritm liniar execută operațiile într-o ordine fixă: citește datele, evaluează expresia și afișează rezultatul. În Java, atribuirea memorează rezultatul unei expresii într-o variabilă declarată. Tipul double permite reprezentarea aproximativă a numerelor reale în virgulă mobilă. Din acest motiv, rezultatele transcendente sunt aproximări numerice, nu valori simbolice exacte.

Funcția Math.sin(x) primește un unghi în radiani. Expresia 2 * x reprezintă produsul, iar ridicarea la putere se poate scrie prin înmulțiri sau prin Math.pow. Semnul ^ nu este operator de ridicare la putere în Java. Parantezele sunt obligatorii pentru a păstra structura fracției din enunț.

Numitorul se descompune astfel: x³ − 2x = x(x² − 2) = x(x − √2)(x + √2). Domeniul matematic este mulțimea numerelor reale fără valorile 0, √2 și −√2. Sinusul este definit pentru orice număr real, deci nu impune restricții suplimentare. În program, valorile double corespunzătoare rădăcinilor sunt respinse explicit. Pentru vecinătăți foarte apropiate de rădăcini pot apărea valori mari și pierderi de precizie.

Validările protejează calculul, fără a schimba formula cerută. Double.isFinite respinge NaN și infinitul. Nu folosim un prag epsilon arbitrar pentru a declara nule toate numerele mici: un asemenea prag ar elimina și valori matematic valide. Dacă produsul intermediar sau rezultatul nu poate fi reprezentat finit, programul raportează limita numerică.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

1. Se citește x și se verifică formatul numeric și caracterul finit. 2. Se resping zerourile numitorului. 3. Se calculează numitorul factorizat și expresia. 4. Se verifică limitele double și se afișează 12 zecimale.

Demonstrație pentru x = 1: sin(1) ≈ 0,8414709848078965, numărătorul este 2,8414709848078965, iar numitorul este 1 − 2 = −1. Rezultă y ≈ −2,8414709848078965. Programul afișează −2,841470984808.

Pentru x = −1, atât numărătorul, cât și numitorul își schimbă semnul. Prin urmare y(−1) = y(1), confirmat de a doua rulare. Pentru x = 0, numitorul este nul și nu există o valoare definită. Complexitate: timp O(1), memorie suplimentară O(1).

Planul de testare și rezultatele

Cazul 1: 1. Verificare: „y = -2.841470984808”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: -1. Verificare: „y = -2.841470984808”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 0. Verificare: „numitor nul”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 1.4142135623730951. Verificare: „numitor nul”. Rezultat: trecut; cod de ieșire 0.

Cazul 5: abc. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Cazul 6: 1e200. Verificare: „limitele”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab01.java

```
import java.util.Locale;
import java.util.Scanner;

public class Lab01 {
    public static void main(String[] args) {
        Locale.setDefault(Locale.ROOT);
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 1, varianta 1");
        System.out.println("Introduceți x (în radiani):");
        if (!in.hasNextDouble()) {
            System.out.println("Eroare: introduceți un număr real valid.");
            return;
        }
        double x = in.nextDouble();
        if (!Double.isFinite(x)) {
            System.out.println("Eroare: x trebuie să fie finit.");
            return;
        }
        // x^3 - 2x = x(x - sqrt(2))(x + sqrt(2)).
        double rad = Math.sqrt(2.0);
        if (x == 0.0 || x == rad || x == -rad) {
            System.out.println("Eroare: numitor nul; expresia nu este definită.");
            return;
        }
        double numitor = x * (x - rad) * (x + rad);
        double y = (Math.sin(x) + 2.0 * x) / numitor;
        if (!Double.isFinite(numitor) || !Double.isFinite(y)
                || numitor == 0.0) {
            System.out.println("Eroare: limitele reprezentării double au fost depășite.");
            return;
        }
        System.out.printf("y = %.12f%n", y);
    }
}
```

Capturi de ecran și interpretare

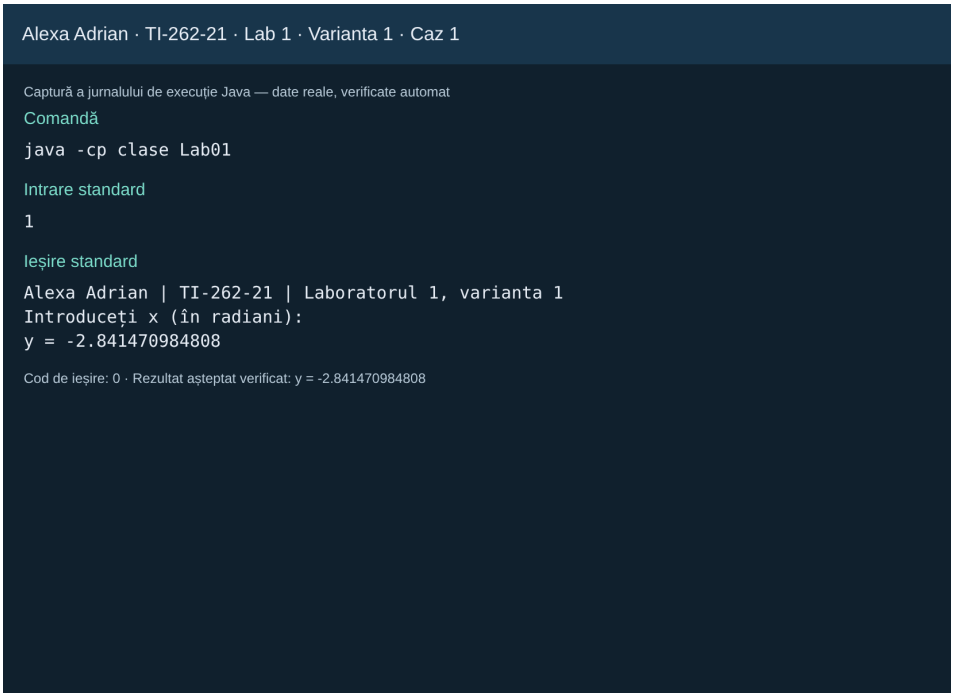


Figura 1. Cazul 1: rezultatul așteptat „y = -2.841470984808” a fost confirmat de execuție.

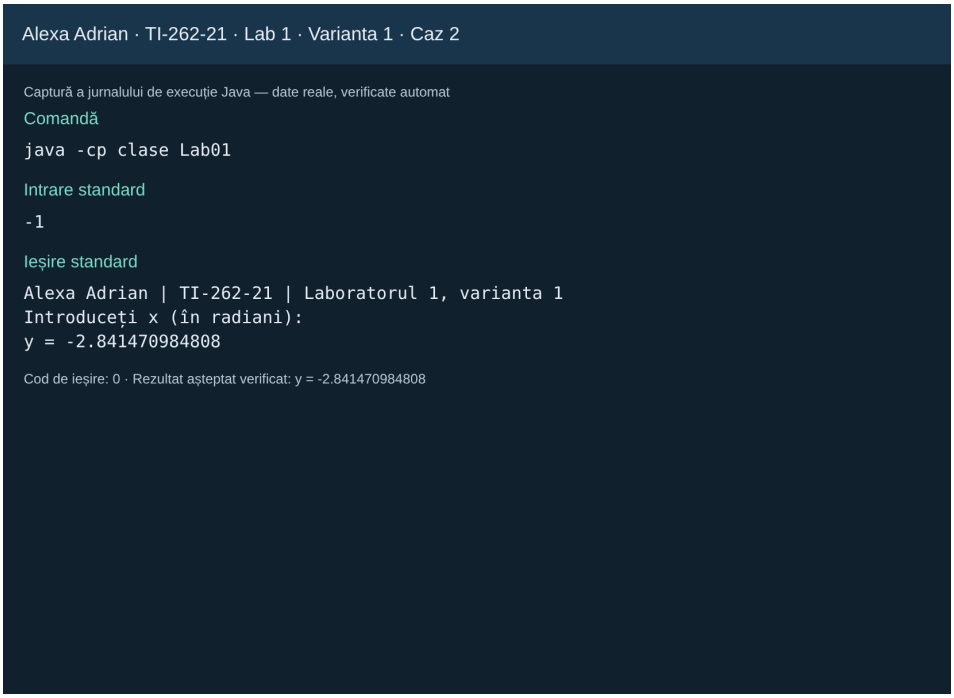


Figura 2. Cazul 2: rezultatul așteptat „y = -2.841470984808” a fost confirmat de execuție.

Capturi de ecran și interpretare



Figura 3. Cazul 3: rezultatul așteptat „numitor nul” a fost confirmat de execuție.

Concluzii

1. Factorizarea numitorului permite determinarea exactă a domeniului matematic.

2. Rulările pentru x = 1 și x = −1 confirmă simetria și rezultatul calculat manual.

3. Verificarea formatului și a valorilor nefinite evită afișarea unui rezultat numeric înșelător.

Întrebări pentru pregătirea susținerii

• De ce Math.sin folosește radiani?

• De ce nu scriem x ^ 3?

• Ce se întâmplă lângă √2?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab01.java, apoi java -cp clase Lab01. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.